

Manual Técnico

Título: Notebook de Selección de Pictogramas Multimodal
con CLIP

Autor: Azucena Abigail Vargas Hernández

Versión: 1.0

Fecha: 02/octubre/2025

Contenido

Manual Técnico: Notebook de Selección de Pictogramas Multimodal	3
Control de cambios	3
Resumen y alcance.....	3
Arquitectura y dependencias.....	3
Entorno e instalación	3
Estructura del código	4
Convenciones	4
Detalle de módulos	5
Pruebas.....	5
Rendimiento y seguridad.....	6
Mantenimiento.....	6
Descripción de cada Bloque del Notebook	6
Bloque 1- Instalar Dependencias	6
Bloque 2: Imports y Configuración.....	7
Bloque 3: Utilidades de Texto y Labels	10
Bloque 4: Carga de Entradas y Modelos	14
Bloque 5: Heurísticas Multimodales	18
Bloque 6: Parámetros de Contexto y Utilidades de Selección.....	25
Bloque 7: Selector por Token (Función Principal)	28
Bloque 8: Bucle Principal y Guardado	33
Glosario y referencias	35
Referencias	36

Manual Técnico: Notebook de Selección de Pictogramas Multimodal

Control de cambios

Versión	Fecha	Autor	Descripción del cambio
1.0	02/oct/2025	Abigail Vargas	Versión inicial del sistema multimodal

Resumen y alcance

¿Qué hace el software?

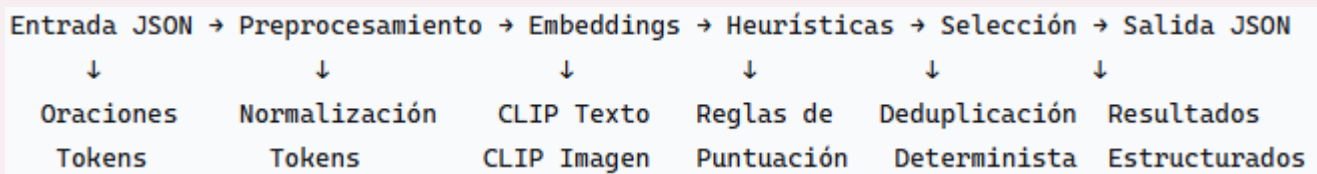
- Procesa oraciones en español y selecciona pictogramas apropiados para cada token
- Utiliza modelos CLIP multilingües para calcular similitudes texto-imagen
- Aplica reglas heurísticas y deduplicación para mejorar la precisión
- Genera un JSON estructurado con las selecciones finales

¿Qué NO hace?

- No genera nuevos pictogramas
- No entrena modelos desde cero
- No maneja idiomas distintos al español
- No procesa video o audio

Arquitectura y dependencias

Diagrama de flujo



Librerías/SDKs y versiones

- sentence-transformers >= 2.2.0
- pillow >= 8.0.0
- rapidfuzz >= 1.9.0
- unidecode >= 1.3.0
- numpy >= 1.20.0

Entorno e instalación

Requisitos:

- Python 3.7+

- 4GB RAM mínimo
- 2GB espacio en disco para modelos
- Conexión a internet para descargar modelos

Instalación

`pip install -q -U sentence-transformers pillow rapidfuzz unidecode`

Verificar estructura de archivos

```
/content/
├─ candidatos_salud_v3.json
└─ todos_pictogramas/
```

Estructura de entrada esperada (json):

```
{
  "oraciones": [
    {
      "oracion": "texto de ejemplo",
      "tokens": ["token1", "token2"],
      "candidatos": {"token1": ["img1.jpg", "img2.jpg"]}
    }
  ]
}
```

Estructura del código

Árbol de módulos

```
multimodal_system/
├─ config/          # Parámetros y constantes
├─ text_utils/      # Normalización y procesamiento
├─ embedding_utils/ # Modelos CLIP y cache
├─ heuristics/      # Reglas de puntuación
├─ selection/       # Lógica de selección
└─ main.py         # Bucle principal
```

Convenciones

Estilo de código:

- Snake_case para funciones y variables
- Tipado en funciones principales
- Docstrings en formato Google
- Constantes en MAYÚSCULAS

Manejo de errores

- Validación de archivos de entrada
- Cache para embeddings
- Fallback en casos de duplicados

Detalle de módulos

Bloque 1: Instalación de Dependencias

- Objetivo: Configurar el entorno con las librerías esenciales

Bloque 2: Imports y Configuración

- Objetivo: Inicializar constantes, rutas y parámetros del sistema

Bloque 3: Utilidades de Texto y Labels

- Objetivo: Proporcionar funciones para normalización y transformación textual

Bloque 4: Carga de Entradas y Modelos

- Objetivo: Cargar datos, modelos CLIP e implementar sistemas de cache

Bloque 5: Heurísticas Multimodales

- Objetivo: Implementar reglas de bonos/penalizaciones para refinar selecciones

Bloque 6: Parámetros de Contexto y Utilidades

- Objetivo: Establecer pesos de combinación y funciones de selección determinista

Bloque 7: Selector por Token

- Objetivo: Algoritmo principal que combina embeddings y heurísticas para selección

Bloque 8: Bucle Principal

- Objetivo: Orquestar el procesamiento completo y generar resultados finales

Pruebas

Datos de prueba

```
{
  "oracion": "El paciente tiene dolor de cabeza",
  "tokens": ["El", "paciente", "tiene", "dolor", "de", "cabeza"],
  "candidatos": {
    "dolor": ["7156-dolor_de_cabeza.jpg", "7157-dolor_abdominal.jpg"],
    "cabeza": ["8201-cabeza_humana.jpg", "8202-cabeza_radiografia.jpg"]
  }
}
```

Criterios de aceptación

- Selecciona pictogramas relevantes para tokens con contenido
- Evita duplicados en misma oración
- Maneja tokens funcionales correctamente

- Genera JSON estructurado consistente

Rendimiento y seguridad

Optimizaciones

- Cache de embeddings para evitar recálculos
- Procesamiento por lotes en modelos CLIP
- Deduplicación en memoria

Límites conocidos

- 64 batch size para texto, 32 para imágenes
- Dependencia de modelos preentrenados
- Consumo de memoria con muchos candidatos

Mantenimiento

Tareas recurrentes

- Actualizar modelos CLIP periódicamente (deben coincidir ambos en dimensión para que hagan match de texto e imagen)
- Revisar y ajustar parámetros heurísticos
- Validar nuevos pictogramas añadidos

Actualización de dependencias

- `pip install -U sentence-transformers pillow`

Descripción de cada Bloque del Notebook

Bloque 1- Instalar Dependencias

Propósito del bloque: Preparar el entorno para embeddings con Sentence-Transformers y utilidades de procesamiento de texto e imagen.

```
!pip -q install -U sentence-transformers pillow rapidfuzz unidecode
```

- `!pip`: Ejecuta el gestor de paquetes de Python (pip) desde el entorno. El símbolo `!` permite ejecutar comandos del sistema en entornos como Jupyter Notebook.
- `-q`: Modo silencioso (quiet). Suprime mensajes innecesarios durante la instalación, mostrando solo resultados críticos.
- `Install`: Comando de pip para instalar paquetes listados a continuación.
- `-U`: Actualiza (Upgrade) los paquetes a su versión más reciente si ya están instalados.
- `sentence-transformers`: Instala la librería para generar embeddings de texto usando modelos de Transformer (en este caso CLIP).
- `Pillow`: Instala la versión actualizada de PIL (Python Imaging Library), utilizada para procesamiento de imágenes.
- `Rapidfuzz`: Proporciona herramientas para comparación de cadenas de texto de forma eficiente (búsqueda difusa).

- unicode: Convierte texto Unicode a una representación ASCII aproximada, normalizando caracteres especiales (ej: "café" → "cafe").

Entradas: ninguna.

Salidas: paquetes instalados/actualizados en el entorno del notebook, actualización de torch/transformers como dependencias de sentence-transformers.

Bloque 2: Imports y Configuración

Propósito del bloque: Inicializar y configurar todo el entorno de ejecución, definiendo las constantes, rutas, modelos y parámetros que serán utilizados por los subsistemas de procesamiento de texto e imágenes en los bloques siguientes.

```
# =====
# 1) Imports y Config
# =====

import os, re, json, math, unicodedata, csv, hashlib
from pathlib import Path
from collections import defaultdict

import numpy as np
from PIL import Image
from sentence_transformers import SentenceTransformer
from unicode import unicode

# --- Rutas
JSON_PATH = "/content/candidatos_salud_v3.json" # JSON de candidatos
PICTOS_DIR = "/content/todos_pictogramas" # Carpeta con imágenes
OUT_JSON = "/content/selecciones_saludnuevo_v3_multimodal_estructurado.json" # salida

# --- Modelos CLIP (texto/imágenes)
TEXT_MODEL_NAME = "sentence-transformers/clip-ViT-B-32-multilingual-v1" # M-CLIP (texto)
IMAGE_MODEL_NAME = "clip-ViT-B-32" # CLIP visión

# --- Heurísticas (bonos/penalizaciones ligeras)
BONUS_MATCH_EXACTO = 0.06
BONUS_SUBCADENA = 0.03
BONUS_UNIPALABRA = 0.03
BONUS_FRASE_NGRAM = 0.18 # ej. "dolor de cabeza"
BONUS_SUFFIX_TOKEN = 0.02 # ej. "... de cabeza"
PENALIZA_PREGUNTAS = -0.08
PENALIZA_PROHIBIDO = -0.12

# Evitar falsos (token -> [(fragmento_en_label, penalizacion)])
PENALIZA_CATEGORIA = {
    "fuerte": [("caja_fuerte", -0.40)]
}

# --- Dedupe por oración (no repetir conceptos/archivos dentro de la MISMA oración)
STRICT_NO_DUP = True
FALLBACK_IF_ALL_DUP = True # si todos chocan por concepto, permitir mejor duplicado de concepto (archivo distinto)

# --- Palabras funcionales
INCLUIR_FUNCIONALES = True
FUNCIONALES = set("de del la el los las un una y o u a en con por para sin al del lo le les me te se que".split())
```

Línea de Código	Explicación Técnica
import os, re, json, math, unicodedata, csv, hashlib	Importa librerías estándar de Python: - os: Operaciones del sistema - re: Expresiones

	regulares - json: Manipulación JSON - math: Funciones matemáticas - unicodedata: Manejo de caracteres Unicode - csv: Procesamiento CSV - hashlib: Funciones hash
from pathlib import Path	Importa Path para manejo orientado a objetos de rutas de archivos
from collections import defaultdict	Importa defaultdict para diccionarios con valores por defecto
import numpy as np	Importa NumPy para cálculos numéricos con alias np
from PIL import Image	Importa Image de Pillow para procesamiento de imágenes
from sentence_transformers import SentenceTransformer	Importa el modelo para generar embeddings de texto
from unidecode import unidecode	Importa función para normalizar caracteres Unicode
JSON_PATH = "/content/candidatos_salud_v3.json"	Define ruta del archivo JSON de entrada con datos de candidatos
PICTOS_DIR = "/content/todos_pictogramas"	Define directorio con imágenes de pictogramas

OUT_JSON = "/content/selecciones_saludnuevo_v3_multimodal_estructurado.json"	Define ruta del archivo JSON de salida
TEXT_MODEL_NAME = "sentence-transformers/clip-ViT-B-32-multilingual-v1"	Especifica modelo multilingüe CLIP para procesar texto
IMAGE_MODEL_NAME = "clip-ViT-B-32"	Especifica modelo CLIP para procesar imágenes
BONUS_MATCH_EXACTO = 0.06	Define bono por coincidencia exacta en búsqueda
BONUS_SUBCADENA = 0.03	Define bono por coincidencia parcial (subcadena)
BONUS_UNIPALABRA = 0.03	Define bono por coincidencia de palabra única
BONUS_FRASE_NGRAM = 0.18	Define bono por coincidencia de frases complejas
BONUS_SUFFIX_TOKEN = 0.02	Define bono por coincidencia de sufijos
PENALIZA_PREGUNTAS = -0.08	Define penalización por términos interrogativos
PENALIZA_PROHIBIDO = -0.12	Define penalización por términos prohibidos
PENALIZA_CATEGORIA = {"fuerte": [("caja_fuerte", -0.40)]}	Diccionario de penalizaciones por categorías específicas
STRICT_NO_DUP = True	Habilita eliminación estricta de duplicados en misma oración
FALLBACK_IF_ALL_DUP = True	Permite fallback si todos los resultados son duplicados

INCLUIR_FUNCIONALES = True	Habilita inclusión de palabras funcionales en procesamiento
FUNCIONALES = set("de del la el los las un una y o u a en con por para sin al del lo le les me te se que".split())	Conjunto de palabras funcionales (stop words) para filtrado

Entradas:

- Rutas de archivos predefinidas
(/content/candidatos_salud_v3.json, /content/todos_pictogramas)
- Nombres de modelos preestablecidos de Hugging Face
- Valores numéricos predefinidos para bonos y penalizaciones
- Lista predefinida de palabras funcionales en español

Salidas:

- Módulos y librerías cargados en memoria
- Variables de configuración inicializadas:
 - Rutas de entrada/salida (JSON_PATH, PICTOS_DIR, OUT_JSON)
 - Modelos especificados (TEXT_MODEL_NAME, IMAGE_MODEL_NAME)
 - Parámetros de heurísticas (variables BONUS_*, PENALIZA_*)
 - Configuraciones de deduplicación (STRICT_NO_DUP, FALLBACK_IF_ALL_DUP)
 - Conjunto de palabras funcionales (FUNCIONALES)

Bloque 3: Utilidades de Texto y Labels

Propósito del Bloque: Proveer funciones auxiliares para la normalización y transformación de texto, etiquetas y nombres de archivo, que serán utilizadas en el preprocesamiento y comparación de textos en el sistema.

```

# =====
# 2) Utilidades de texto/labels
# =====
def norm(s: str) -> str:
    s = s.strip().lower()
    s = ''.join(c for c in unicodedata.normalize('NFD', s) if unicodedata.category(c) != 'Mn')
    s = s.replace('¿', '').replace('!', '')
    s = s.replace('_', ' ').replace('-', ' ')
    s = re.sub(r"\s+", " ", s).strip()
    return s

def fname_to_label(fname: str) -> str:
    stem = Path(fname).stem # "7156-dolor_de_cabeza"
    stem = re.sub(r"^\d+-", '', stem) # "dolor_de_cabeza"
    return stem.replace('_', ' ').replace('-', ' ').strip()

def canonical_label_key_from_fname(fname: str) -> str:
    """Clave de concepto canónica: sin ID, sin tildes, sin guiones."""
    stem = Path(fname).stem
    stem = re.sub(r"^\d+-", '', stem)
    s = stem.replace('_', ' ').replace('-', ' ').lower().strip()
    s = ''.join(c for c in unicodedata.normalize('NFD', s) if unicodedata.category(c) != 'Mn')
    s = re.sub(r"\s+", ' ', s)
    return s # ej. "dolor de cabeza"

def tokens_norm(tokens):
    return [norm(t) for t in tokens]

def ngrams(tokens, n):
    return [" ".join(tokens[i:i+n]) for i in range(len(tokens)-n+1)]

def label_norm(label):
    return norm(label)

def contiene_frase(label_n: str, frase_n: str) -> bool:
    return f" {frase_n} " in f" {label_n} "

```

Explicación Función por Función:

1. Función norm:

- **Entrada:** Una cadena de texto s.
- **Proceso:**
 - Convierte a minúsculas y elimina espacios al inicio y final.
 - Normaliza la cadena eliminando tildes y diacríticos (usando descomposición Unicode y removiendo marcas de combinación).
 - Elimina signos de interrogación y exclamación iniciales.
 - Reemplaza guiones bajos y medios por espacios.
 - Reduce múltiples espacios a uno solo.
- **Salida:** Cadena normalizada.

2. Función fname_to_label:

- **Entrada:** Nombre de archivo fname.

- **Proceso:**
 - Extrae el stem (sin extensión) del archivo.
 - Elimina cualquier número al inicio seguido de un guion (ej: "7156-dolor_de_cabeza" -> "dolor_de_cabeza").
 - Reemplaza guiones bajos y medios por espacios.
 - **Salida:** Etiqueta legible.
3. **Función** canonical_label_key_from_fname:
- **Entrada:** Nombre de archivo fname.
 - **Proceso:**
 - Similar a fname_to_label, pero además aplica la normalización (eliminación de tildes y diacríticos) y asegura una clave canónica.
 - **Salida:** Clave canónica para el concepto (ej: "dolor de cabeza").
4. **Función** tokens_norm:
- **Entrada:** Lista de tokens (cadenas).
 - **Proceso:** Aplica la función norm a cada token de la lista.
 - **Salida:** Lista de tokens normalizados.
5. **Función** ngrams:
- **Entrada:** Lista de tokens y un entero n (tamaño del n-grama).
 - **Proceso:** Genera una lista de n-gramas (secuencias de n tokens consecutivos) a partir de la lista de tokens.
 - **Salida:** Lista de n-gramas (cadenas).
6. **Función** label_norm:
- **Entrada:** Una etiqueta (cadena).
 - **Proceso:** Aplica la función norm a la etiqueta.
 - **Salida:** Etiqueta normalizada.
7. **Función** contiene_frase:
- **Entrada:** Una etiqueta normalizada label_n y una frase normalizada frase_n.
 - **Proceso:** Comprueba si la frase está contenida en la etiqueta, agregando espacios al inicio y final para coincidencias exactas de palabras.
 - **Salida:** Booleano (True si la frase está contenida).

Línea de Código

Explicación Técnica

def norm(s: str) -> str:	Define función de normalización textual con tipado
s = s.strip().lower()	Elimina espacios y convierte a minúsculas
s = ".join(c for c in unicodedata.normalize('NFD', s) if unicodedata.category(c) != 'Mn')	Elimina tildes y diacríticos usando descomposición Unicode
s = s.replace('¿','').replace('!', '')	Elimina signos de interrogación/exclamación iniciales
s = s.replace('_', ' ').replace('-', ' ')	Reemplaza guiones bajos y medios por espacios
s = re.sub(r"\s+", " ", s).strip()	Normaliza espacios múltiples a espacio simple
def fname_to_label(fname: str) -> str:	Convierte nombre de archivo a etiqueta legible
stem = Path(fname).stem	Extrae nombre del archivo sin extensión
stem = re.sub(r'^\d+-', '', stem)	Elimina prefijo numérico (ej: "7156-")
return stem.replace('_', ' ').replace('-', ' ').strip()	Reemplaza separadores por espacios
def canonical_label_key_from_fname(fname: str) -> str:	Crea clave canónica para comparación
"""Clave de concepto canónica: sin ID, sin tildes, sin guiones."""	Documentación: clave unificada
stem = Path(fname).stem	Obtiene stem del archivo
stem = re.sub(r'^\d+-', '', stem)	Remueve prefijo numérico
s = stem.replace('_', ' ').replace('-', ' ').lower().strip()	Normaliza separadores y case
s = ".join(c for c in unicodedata.normalize('NFD', s) if unicodedata.category(c) != 'Mn')	Elimina caracteres diacríticos
s = re.sub(r'\s+', ' ', s)	Normaliza espacios
def tokens_norm(tokens):	Normaliza lista de tokens
return [norm(t) for t in tokens]	Aplica normalización a cada token
def ngrams(tokens, n):	Genera n-gramas de tokens

return [" ".join(tokens[i:i+n]) for i in range(len(tokens)-n+1)]	Crea combinaciones de n tokens consecutivos
def label_norm(label):	Función wrapper para normalizar etiquetas
return norm(label)	Aplica función de normalización
def contiene_frase(label_n: str, frase_n: str) -> bool:	Detecta frases dentro de texto normalizado
return f" {frase_n} " in f" {label_n} "	Busca frase con delimitación por espacios

Entradas del Bloque (Globales):

- Las funciones definidas en este bloque no se ejecutan inmediatamente, sino que se definen para su uso posterior.
- Dependen de las librerías y módulos importados en el bloque anterior (como unicodedata, re, Path).

Salidas del Bloque:

- Siete funciones de utilidad para el procesamiento de texto, que estarán disponibles para los bloques siguientes. Funciones de utilidad para procesamiento de texto:
 - norm() - Normalización de texto
 - fname_to_label() - Conversión de nombre de archivo a etiqueta
 - canonical_label_key_from_fname() - Generación de clave canónica
 - tokens_norm() - Normalización de tokens
 - ngrams() - Generación de n-gramas
 - label_norm() - Normalización de etiquetas
 - contiene_frase() - Detección de frases en texto

Bloque 4: Carga de Entradas y Modelos

Propósito del Bloque: Cargar y verificar todos los recursos necesarios (datos, modelos), implementar funciones eficientes para generar embeddings de texto e imágenes, y establecer sistemas de cache para optimizar el rendimiento evitando recálculos redundantes durante el proceso de búsqueda multimodal.

```

# =====
# 3) Carga entradas / modelos
# =====
if not os.path.exists(JSON_PATH):
    raise FileNotFoundError(f"No encuentro {JSON_PATH}. Sube el JSON o monta Drive.")
if not os.path.isdir(PICTOS_DIR):
    raise FileNotFoundError(f"No existe la carpeta de pictogramas: {PICTOS_DIR}")

data = json.load(open(JSON_PATH, "r", encoding="utf-8"))
oraciones = data.get("oraciones", [])
print("Oraciones cargadas:", len(oraciones))

text_model = SentenceTransformer(TEXT_MODEL_NAME) # M-CLIP: texto
image_model = SentenceTransformer(IMAGE_MODEL_NAME) # CLIP: imagen

def emb_text(texts):
    return text_model.encode(
        texts,
        convert_to_numpy=True,
        normalize_embeddings=True,
        show_progress_bar=False,
        batch_size=64
    )

def emb_images(pil_list):
    return image_model.encode(
        pil_list,
        convert_to_numpy=True,
        normalize_embeddings=True,
        show_progress_bar=False,
        batch_size=32
    )

# --- Caches
img_vec_cache = [] # fname -> vec
text_vec_cache = [] # *** clave: TEXTO ORIGINAL ***
label_cache = [] # fname -> label bonito

def get_text_vec(s: str):
    key = s
    if key not in text_vec_cache:
        text_vec_cache[key] = emb_text([s])[0]
    return text_vec_cache[key]

def get_label(fname: str):
    if fname not in label_cache:
        label_cache[fname] = fname_to_label(fname)
    return label_cache[fname]

def get_image_vec(fname: str):
    if fname in img_vec_cache:
        return img_vec_cache[fname]
    path = os.path.join(PICTOS_DIR, fname)
    if not os.path.exists(path):
        raise FileNotFoundError(f"No existe la imagen: {path}")
    img = Image.open(path).convert("RGB")
    vec = emb_images([img])[0]
    img_vec_cache[fname] = vec
    return vec

```

Línea de Código	Explicación Técnica
if not os.path.exists(JSON_PATH):	Verifica existencia del archivo JSON de entrada
raise FileNotFoundError(f'No encuentro {JSON_PATH}...')	Lanza error si no encuentra el archivo JSON

if not os.path.isdir(PICTOS_DIR):	Verifica existencia del directorio de pictogramas
raise FileNotFoundError(f"No existe la carpeta...")	Lanza error si no encuentra el directorio de imágenes
data = json.load(open(JSON_PATH, "r", encoding="utf-8"))	Carga y parsea el archivo JSON con codificación UTF-8
oraciones = data.get("oraciones", [])	Extrae lista de oraciones del JSON, usa lista vacía por defecto
print("Oraciones cargadas:", len(oraciones))	Muestra en consola el número de oraciones cargadas
text_model = SentenceTransformer(TEXT_MODEL_NAME)	Instancia modelo multilingüe CLIP para procesar texto
image_model = SentenceTransformer(IMAGE_MODEL_NAME)	Instancia modelo CLIP para procesar imágenes
def emb_text(texts):	Define función para generar embeddings de texto
return text_model.encode(	Ejecuta encoding del modelo de texto
texts, convert_to_numpy=True, normalize_embeddings=True, show_progress_bar=False, batch_size=64	Parámetros: entrada, formato NumPy, normalización, sin barra progreso, lote 64
def emb_images(pil_list):	Define función para generar embeddings de imágenes
return image_model.encode(	Ejecuta encoding del modelo de imágenes
pil_list, convert_to_numpy=True, normalize_embeddings=True, show_progress_bar=False, batch_size=32	Parámetros: lista PIL, formato NumPy, normalización, lote 32 (menor por memoria)
img_vec_cache = {}	Inicializa cache para embeddings de imágenes (nombre → vector)

text_vec_cache = {}	Inicializa cache para embeddings de texto (texto → vector)
label_cache = {}	Inicializa cache para etiquetas procesadas (nombre → label)
def get_text_vec(s: str):	Define función con cache para obtener embedding de texto
key = s	Usa el texto original como clave de cache
if key not in text_vec_cache:	Verifica si ya está en cache
text_vec_cache[key] = emb_text([s])[0]	Calcula y almacena embedding si no está en cache
return text_vec_cache[key]	Retorna embedding desde cache
def get_label(fname: str):	Define función con cache para obtener etiqueta
if fname not in label_cache:	Verifica si etiqueta ya está en cache
label_cache[fname] = fname_to_label(fname)	Calcula y almacena etiqueta si no está en cache
return label_cache[fname]	Retorna etiqueta desde cache
def get_image_vec(fname: str):	Define función con cache para obtener embedding de imagen
if fname in img_vec_cache:	Verifica si embedding ya está en cache
return img_vec_cache[fname]	Retorna embedding desde cache si existe
path = os.path.join(PICTOS_DIR, fname)	Construye ruta completa del archivo de imagen
if not os.path.exists(path):	Verifica existencia del archivo de imagen

raise FileNotFoundError(f"No existe la imagen: {path}")	Lanza error si imagen no existe
img = Image.open(path).convert("RGB")	Abre imagen y convierte a formato RGB estándar
vec = emb_images([img])[0]	Calcula embedding de la imagen
img_vec_cache[fname] = vec	Almacena embedding en cache
return vec	Retorna embedding calculado

Entradas:

- Rutas definidas en el bloque 2 (JSON_PATH, PICTOS_DIR)
- Modelos especificados (TEXT_MODEL_NAME, IMAGE_MODEL_NAME)
- Archivo JSON con datos de oraciones
- Imágenes en el directorio de pictogramas

Salidas:

- data: Diccionario con datos cargados del JSON
- oraciones: Lista de oraciones procesables
- text_model: Modelo de embeddings para texto
- image_model: Modelo de embeddings para imágenes
- Funciones de encoding: emb_text(), emb_images()
- Sistemas de cache: img_vec_cache, text_vec_cache, label_cache
- Funciones de acceso: get_text_vec(), get_label(), get_image_vec()

Bloque 5: Heurísticas Multimodales

Propósito del Bloque: Implementar un sistema de reglas heurísticas que complemente los embeddings multimodales, aplicando bonos por coincidencias textuales específicas y penalizaciones por contextos no deseados, mejorando así la precisión en la selección de pictogramas relevantes.

```

# =====
# 4) Heurísticas multimodales
# =====
def frase_bonus_desde_contexto(label_n: str, toks_n: list) -> float:
    """Bono por n-gramas presentes en la oración; incluye variante 'X de Y'."""
    bonus = 0.0
    bigr = ngrams(toks_n, 2)
    trigr = ngrams(toks_n, 3)
    bigr_de = []
    for b in bigr:
        w = b.split()
        if len(w) == 2:
            bigr_de.append(f"{w[0]} de {w[1]}")
    for frase in trigr + bigr + bigr_de:
        if contiene_frase(label_n, frase):
            bonus = max(bonus, BONUS_FRASE_NGRAM)
    return bonus

def reglas_heuristicas(token_raw, token_n, cand_fname, cand_label_raw, cand_label_n, toks_n):
    score = 0.0
    # Exacto + unipalabra
    if token_n == cand_label_n:
        score += BONUS_MATCH_EXACTO
        if len(cand_label_raw.split()) == 1:
            score += BONUS_UNIPALABRA
    # Subcadena (con límites de palabra)
    if re.search(rf"(?:^\s)(re.escape(token_n))(?:\s|$)", cand_label_n):
        score += BONUS_SUBCADENA
    # Frase compuesta (ngram)
    score += frase_bonus_desde_contexto(cand_label_n, toks_n)
    # Sufijo "- token"
    if cand_label_n.endswith(f" {token_n}"):
        score += BONUS_SUFFIX_TOKEN
    # Penalizaciones generales
    lname = cand_label_n
    has_no = re.search(r"\bno\b", lname) is not None
    if (("prohibido" in lname) or has_no) and token_n not in ("no", "prohibido"):
        score += PENALIZA_PROHIBIDO
    if "?" in cand_label_raw or "." in cand_label_raw:
        score += PENALIZA_PREGUNTAS
    # Penalizaciones ad-hoc
    for clave, pares in PENALIZA_CATEGORIA.items():
        if token_n == clave:
            for fragmento, pena in pares:
                if fragmento in lname:
                    score += pena
    # Preferencia simple de género (roles comunes)
    if token_n.endswith("a") and cand_label_n.endswith("o") and any(w in token_n for w in ["enfermer", "medic"]):
        score -= 0.03
    if token_n.endswith("o") and cand_label_n.endswith("a") and any(w in token_n for w in ["enfermer", "medic"]):
        score -= 0.03
    return score

```

Explicación línea por línea:

1. `def frase_bonus_desde_contexto(label_n: str, toks_n: list) -> float:`
 - Define una función que calcula un bono por la presencia de n-gramas (bigramas, trigramas y una variante con 'de') en la etiqueta normalizada, dados los tokens normalizados de la oración.
2. `"""Bono por n-gramas presentes en la oración; incluye variante 'X de Y'."""`
 - Documentación de la función.
3. `bonus = 0.0`
 - Inicializa el bono en 0.
4. `bigr = ngrams(toks_n, 2)`
 - Genera bigramas a partir de los tokens normalizados.
5. `trigr = ngrams(toks_n, 3)`

- Genera trigramas a partir de los tokens normalizados.
- 6. `bigr_de = []`
 - Inicializa una lista para almacenar bigramas con la preposición 'de' insertada.
- 7. `for b in bigr:`
 - Itera sobre cada bigrama.
- 8. `w = b.split()`
 - Divide el bigrama en palabras.
- 9. `if len(w) == 2:`
 - Verifica que el bigrama tenga exactamente dos palabras.
- 10. `bigr_de.append(f'{w[0]} de {w[1]}')`
 - Crea una frase con la forma "X de Y" y la añade a la lista.
- 11. `for frase in trigr + bigr + bigr_de:`
 - Itera sobre todos los n-gramas (trigramas, bigramas y bigramas con 'de').
- 12. `if contiene_frase(label_n, frase):`
 - Si la etiqueta normalizada contiene la frase (n-grama), entonces:
- 13. `bonus = max(bonus, BONUS_FRASE_NGRAM)`
 - Actualiza el bono al máximo entre el actual y el valor de `BONUS_FRASE_NGRAM`.
- 14. `return bonus`
 - Retorna el bono calculado.
- 15. `def reglas_heuristicas(token_raw, token_n, cand_fname, cand_label_raw, cand_label_n, toks_n):`
 - Define una función que aplica varias reglas heurísticas para calcular un puntaje de similitud entre un token y un candidato.
- 16. `score = 0.0`
 - Inicializa el puntaje en 0.
- 17. `if token_n == cand_label_n:`
 - Si el token normalizado es igual a la etiqueta normalizada del candidato:
- 18. `score += BONUS_MATCH_EXACTO`
 - Añade el bono por coincidencia exacta.
- 19. `if len(cand_label_raw.split()) == 1:`
 - Si la etiqueta cruda del candidato es una sola palabra:

20. score += BONUS_UNIPALABRA

- Añade el bono por palabra única.

21. if re.search(rf"(?:^\s){re.escape(token_n)}(?:\s|\$)", cand_label_n):

- Busca el token normalizado como una palabra completa (no subcadena suelta) en la etiqueta normalizada del candidato.

22. score += BONUS_SUBCADENA

- Añade el bono por subcadena.

23. score += frase_bonus_desde_contexto(cand_label_n, toks_n)

- Añade el bono por frases (n-gramas) calculado por la función anterior.

24. if cand_label_n.endswith(f" {token_n}"):

- Si la etiqueta normalizada del candidato termina con el token normalizado (como última palabra).

25. score += BONUS_SUFFIX_TOKEN

- Añade el bono por sufijo.

26. lname = cand_label_n

- Asigna la etiqueta normalizada a una variable local.

27. has_no = re.search(r"\bno\b", lname) is not None

- Verifica si la etiqueta normalizada contiene la palabra "no".

28. if (("prohibido" in lname) or has_no) and token_n not in {"no", "prohibido"}:

- Si la etiqueta contiene "prohibido" o "no" y el token no es "no" o "prohibido":

29. score += PENALIZA_PROHIBIDO

- Aplica penalización por términos prohibidos.

30. if "?" in cand_label_raw or "¿" in cand_label_raw:

- Si la etiqueta cruda del candidato contiene signos de interrogación:

31. score += PENALIZA_PREGUNTAS

- Aplica penalización por preguntas.

32. for clave, pares in PENALIZA_CATEGORIA.items():

- Itera sobre las categorías de penalizaciones definidas.

33. if token_n == clave:

- Si el token normalizado coincide con la clave de penalización:

34. for fragmento, pena in pares:

- Itera sobre los fragmentos y penalizaciones de esa categoría.

35. if fragmento in lname:

- Si el fragmento está en la etiqueta normalizada:

36. score += pena

- Aplica la penalización.

37. if token_n.endswith("a") and cand_label_n.endswith("o") and any(w in token_n for w in ["enfermer", "medic"]):

- Si el token termina en "a" (femenino) y el candidato termina en "o" (masculino) y el token contiene "enfermer" o "medic":

38. score -= 0.03

- Penalización leve por discordancia de género (femenino vs masculino).

39. if token_n.endswith("o") and cand_label_n.endswith("a") and any(w in token_n for w in ["enfermer", "medic"]):

- Si el token termina en "o" (masculino) y el candidato termina en "a" (femenino) y el token contiene "enfermer" o "medic":

40. score -= 0.03

- Penalización leve por discordancia de género (masculino vs femenino).

41. return score

- Retorna el puntaje calculado.

Línea de Código	Explicación Técnica
def frase_bonus_desde_contexto(label_n: str, toks_n: list) -> float:	Define función para calcular bonos por frases en contexto
"""Bono por n-gramas presentes en la oración; incluye variante 'X de Y'."""	Documentación: bonos por n-gramas y patrones "de"
bonus = 0.0	Inicializa acumulador de bonos
bigr = ngrams(toks_n, 2)	Genera bigramas (2 palabras consecutivas) de tokens
trigr = ngrams(toks_n, 3)	Genera trigramas (3 palabras consecutivas) de tokens
bigr_de = []	Inicializa lista para patrones "X de Y"
for b in bigr:	Itera sobre cada bigrama

w = b.split()	Divide bigrama en palabras individuales
if len(w) == 2:	Verifica que sea exactamente 2 palabras
bigr_de.append(f'{w[0]} de {w[1]}')	Crea patrón "X de Y" y lo agrega a lista
for frase in trigr + bigr + bigr_de:	Itera sobre todos los patrones generados
if contiene_frase(label_n, frase):	Verifica si la etiqueta contiene el patrón
bonus = max(bonus, BONUS_FRASE_NGRAM)	Asigna el bono máximo encontrado
def reglas_heuristicas(token_raw, token_n, cand_fname, cand_label_raw, cand_label_n, toks_n):	Define función principal de heurísticas con múltiples parámetros
score = 0.0	Inicializa puntuación acumulativa
if token_n == cand_label_n:	Verifica coincidencia exacta entre token y etiqueta
score += BONUS_MATCH_EXACTO	Aplica bono por coincidencia exacta
if len(cand_label_raw.split()) == 1:	Verifica si etiqueta es de una sola palabra
score += BONUS_UNIPALABRA	Aplica bono adicional por palabra única
`if re.search(rf"(?:^\s){re.escape(token_n)}(?:\s\$)", cand_label_n):`	Busca token como palabra completa en etiqueta
score += BONUS_SUBCADENA	Aplica bono por coincidencia de subcadena
score += frase_bonus_desde_contexto(cand_label_n, toks_n)	Suma bonos por n-gramas del contexto
if cand_label_n.endswith(f' {token_n}')	Verifica si token aparece al final de la etiqueta
score += BONUS_SUFFIX_TOKEN	Aplica bono por coincidencia de sufijo

lname = cand_label_n	Asigna alias para código más legible
has_no = re.search(r"\bno\b", lname) is not None	Detecta presencia de palabra "no"
if (("prohibido" in lname) or has_no) and token_n not in {"no", "prohibido"}:	Verifica términos negativos cuando token no es negativo
score += PENALIZA_PROHIBIDO	Aplica penalización por términos prohibidos
if "?" in cand_label_raw or "¿" in cand_label_raw:	Detecta signos de interrogación
score += PENALIZA_PREGUNTAS	Aplica penalización por preguntas
for clave, pares in PENALIZA_CATEGORIA.items():	Itera sobre categorías de penalización
if token_n == clave:	Verifica si token coincide con categoría
for fragmento, pena in pares:	Itera sobre fragmentos penalizados
if fragmento in lname:	Verifica si fragmento está en etiqueta
score += pena	Aplica penalización específica
if token_n.endswith("a") and cand_label_n.endswith("o") and any(w in token_n for w in ["enfermer", "medic"]):	Detecta discordancia de género femenino→masculino en profesiones
score -= 0.03	Penalización leve por discordancia de género
if token_n.endswith("o") and cand_label_n.endswith("a") and any(w in token_n for w in ["enfermer", "medic"]):	Detecta discordancia de género masculino→femenino en profesiones
return score	Retorna puntuación heurística total calculada

Entradas:

- Tokens normalizados y en crudo
- Etiquetas de candidatos normalizadas y en crudo
- Nombres de archivo de candidatos
- Listas de tokens normalizados del contexto

- Constantes de bonos/penalizaciones del bloque 2

Salidas:

- frase_bonus_desde_contexto(): Función que calcula bonos por n-gramas
- reglas_heuristicas(): Función que aplica múltiples reglas de puntuación
- Puntuaciones numéricas (flotantes) que representan ajustes de similitud

Bloque 6: Parámetros de Contexto y Utilidades de Selección

Propósito del Bloque: Establecer los parámetros de ponderación para combinar múltiples fuentes de similitud multimodal y proporcionar funciones utilitarias para extraer contexto semántico y realizar selecciones deterministas en casos de empate, garantizando consistencia en los resultados.

```
# =====
# 5) Parámetros NUEVOS (contexto fuerte) + Utilidades
# =====
# Rebalanceo para que el contexto de la ORACIÓN pese de verdad
W_SENT = 0.65 # imagen + oración completa
W_PROMPT = 0.20 # imagen + "pictograma de <token> en contexto ..."
W_TOK2 = 0.15 # imagen + token aislado
W_PHRASE = 0.12 # imagen + "<token> <ancla>" (si hay ancla)

# Empates: si varios quedan dentro de este epsilon del mejor,
# elegimos uno de ese subgrupo de forma determinista POR ORACIÓN.
EPS_SELECT = 0.02

def extraer_ancla_semantica(tokens_raw, i):
    """
    Devuelve una palabra de contenido cercana al token i (o None).
    Prioriza derecha hasta 3 posiciones; si no, izquierda hasta 3.
    Ignora funcionales definidas en FUNCIONALES.
    """
    def es_contenido(t):
        tn = norm(t)
        return tn and (tn not in FUNCIONALES) and len(tn) > 2
    for j in range(i+1, min(i+4, len(tokens_raw))):
        if es_contenido(tokens_raw[j]):
            return tokens_raw[j]
    for j in range(i-1, max(-1, i-4), -1):
        if es_contenido(tokens_raw[j]):
            return tokens_raw[j]
    return None

def pick_determinista_por_oracion(oracion_raw, candidatos, eps=EPS_SELECT):
    """
    candidatos: lista de tuplas (score, paquete)
    Devuelve el paquete ganador. Si hay varios dentro de (mejor_score - eps),
    escoge uno de ese subgrupo de manera determinista usando el hash de la ORACIÓN.
    """
    if not candidatos:
        return None
    candidatos = sorted(candidatos, key=lambda x: x[0], reverse=True)
    best = candidatos[0][0]
    pool = [c for c in candidatos if c[0] >= best - eps]
    if len(pool) == 1:
        return pool[0][1]
    h = hashlib.md5(oracion_raw.encode("utf-8")).hexdigest()
    idx = int(h[:8], 16) % len(pool)
    return pool[idx][1]
```

- Define los pesos para combinar diferentes similitudes:
 - **W_SENT**: Peso para la similitud entre la imagen y la oración completa.
 - **W_PROMPT**: Peso para la similitud entre la imagen y un prompt contextualizado.
 - **W_TOK2**: Peso para la similitud entre la imagen y el token aislado.
 - **W_PHRASE**: Peso para la similitud entre la imagen y una frase que combina el token y un ancla semántica.
- **EPS_SELECT**: Umbral de diferencia para considerar empates. Si las puntuaciones de varios candidatos están dentro de este margen del mejor, se elige uno de ellos de forma determinista.
- **extraer_ancla_semantica**: Función que busca una palabra de contenido (no funcional y con más de 2 caracteres) cerca del token en la posición i.
 - Primero busca a la derecha (hasta 3 posiciones).
 - Si no encuentra, busca a la izquierda (hasta 3 posiciones).
 - Retorna la palabra de contenido encontrada o None si no hay.
- **pick_determinista_por_oracion**: Función que elige un candidato de una lista de candidatos (cada uno con un score y un paquete de datos).
 - Si no hay candidatos, retorna None.
 - Ordena los candidatos por score descendente.
 - Toma el mejor score y forma un grupo con todos los candidatos que tengan un score dentro del margen eps del mejor.
 - Si solo hay uno en el grupo, retorna ese.
 - Si hay varios, usa un hash MD5 de la oración en crudo para seleccionar un índice de manera determinista (siempre igual para la misma oración) y retorna el candidato en esa posición.

Línea de Código	Explicación Técnica
W_SENT = 0.65	Peso principal para similitud con oración completa (65%)
W_PROMPT = 0.20	Peso para similitud con prompt contextualizado (20%)
W_TOK2 = 0.15	Peso para similitud con token aislado (15%)
W_PHRASE = 0.12	Peso para similitud con frase token+ancla (12%)
EPS_SELECT = 0.02	Umbral para considerar empates (2% de diferencia)
def extraer_ancla_semantica(tokens_raw, i):	Define función para extraer palabra ancla contextual
""""Devuelve una palabra de contenido cercana al token i (o None).""""	Documentación: busca palabra significativa cercana

def es_contenido(t):	Define función interna para detectar palabras con contenido
tn = norm(t)	Normaliza el token
return tn and (tn not in FUNCIONALES) and len(tn) > 2	Verifica: no vacío, no funcional, más de 2 caracteres
for j in range(i+1, min(i+4, len(tokens_raw))):	Busca hacia adelante (hasta 3 posiciones)
if es_contenido(tokens_raw[j]):	Verifica si es palabra con contenido
return tokens_raw[j]	Retorna primera palabra con contenido encontrada
for j in range(i-1, max(-1, i-4), -1):	Busca hacia atrás (hasta 3 posiciones)
if es_contenido(tokens_raw[j]):	Verifica si es palabra con contenido
return tokens_raw[j]	Retorna primera palabra con contenido encontrada
return None	Retorna None si no encuentra ancla
def pick_determinista_por_oracion(oracion_raw, candidatos, eps=EPS_SELECT):	Define función de selección determinista
"""candidatos: lista de tuplas (score, paquete)"""	Documentación: estructura de entrada
if not candidatos:	Verifica si no hay candidatos
return None	Retorna None si lista vacía
candidatos = sorted(candidatos, key=lambda x: x[0], reverse=True)	Ordena candidatos por score descendente
best = candidatos[0][0]	Obtiene el mejor score
pool = [c for c in candidatos if c[0] >= best - eps]	Crea grupo de candidatos dentro del umbral de empate
if len(pool) == 1:	Verifica si solo hay un candidato en el grupo
return pool[0][1]	Retorna el único candidato

<code>h = hashlib.md5(oracion_raw.encode("utf-8")).hexdigest()</code>	Genera hash MD5 de la oración para determinismo
<code>idx = int(h[:8], 16) % len(pool)</code>	Convierte primeros 8 caracteres hex a entero y calcula índice
<code>return pool[idx][1]</code>	Retorna candidato seleccionado determinísticamente

Entradas:

- Tokens en crudo de la oración
- Índice del token actual
- Lista de candidatos con puntuaciones
- Parámetros de peso predefinidos
- Conjunto de palabras funcionales del bloque 2

Salidas:

- 4 parámetros de peso para combinación multimodal (W_SENT, W_PROMPT, W_TOK2, W_PHRASE)
- Umbral de selección (EPS_SELECT)
- Función `extraer_ancla_semantica()` para encontrar palabras de contexto relevantes
- Función `pick_determinista_por_oracion()` para selección estable en empates

Bloque 7: Selector por Token (Función Principal)

Implementar el algoritmo principal de selección que combina embeddings multimodales, reglas heurísticas y mecanismos de deduplicación para elegir el pictograma más apropiado para cada token dentro del contexto de una oración completa.

```

def escoger_candidato(token_raw, oracion_raw, candidatos_fnames, toks_n,
                      elegidos_file_set, elegidos_label_set,
                      tokens_raw_list, token_index):

    token_n = norm(token_raw)
    # Funcionales
    if (not INCLUIR_FUNCIONALES) and (token_n in FUNCIONALES):
        return None, {"razon": "token_funcional_descartado"}
    if not candidatos_fnames:
        return None, {"razon": "sin_candidatos"}

    # Embeddings de texto
    sent_vec = get_text_vec(oracion_raw) # oración "pura"
    prompt_vec = get_text_vec(f"Pictograma de {token_raw} en el contexto: {oracion_raw}")
    tok_vec = get_text_vec(token_raw)

    # Ancla semántica (si existe)
    anchor = extraer_ancla_semantica(tokens_raw_list, token_index)
    anchor_vec = get_text_vec(f"{token_raw} {anchor}") if anchor else None

    # Candidatos válidos (dedupe archivo y concepto dentro de la ORACIÓN)
    cand_list = []
    for fname in candidatos_fnames:
        if fname in elegidos_file_set:
            continue # no repetir ARCHIVO en la oración
        try:
            c_vec = get_image_vec(fname)
        except FileNotFoundError:
            continue
        label_raw = get_label(fname)
        label_n = label_norm(label_raw)
        label_key = canonical_label_key_from_fname(fname)
        if STRICT_NO_DUP and (label_key in elegidos_label_set):
            continue # no repetir CONCEPTO en la oración
        cand_list.append((fname, c_vec, label_raw, label_n, label_key))

    # Fallback: permitir duplicar CONCEPTO si no quedó nada (pero sin repetir ARCHIVO)
    if not cand_list and FALLBACK_IF_ALL_DUP:
        for fname in candidatos_fnames:
            if fname in elegidos_file_set:
                continue
            try:
                c_vec = get_image_vec(fname)
            except FileNotFoundError:
                continue
            label_raw = get_label(fname)
            label_n = label_norm(label_raw)
            label_key = canonical_label_key_from_fname(fname)
            cand_list.append((fname, c_vec, label_raw, label_n, label_key))
    if not cand_list:
        return None, {"razon": "sin_candidatos_validos"}

    # Puntuar y seleccionar
    scored = [] # (score_total, paquete)
    for fname, c_vec, label_raw, label_n, label_key in cand_list:
        s_sent = float(np.dot(sent_vec, c_vec)) # oración = imagen
        s_prompt = float(np.dot(prompt_vec, c_vec)) # prompt = imagen
        s_tok2 = float(np.dot(tok_vec, c_vec)) # token = imagen
        s_phrase = float(np.dot(anchor_vec, c_vec)) if anchor_vec is not None else 0.0

        score = (W_SENT * s_sent) + (W_PROMPT * s_prompt) + (W_TOK2 * s_tok2)
        if anchor_vec is not None:
            score += W_PHRASE * s_phrase

        score += reglas_heuristicas(token_raw, token_n, fname, label_raw, label_n, toks_n)
        scored.append((score, (fname, label_raw, label_key)))

    ganador = pick_determinista_por_oracion(oracion_raw, scored, eps=EPS_SELECT)
    if ganador is None:
        return None, {"razon": "sin_candidatos_validos"}

    fname, label_raw, label_key = ganador
    elegidos_file_set.add(fname)
    elegidos_label_set.add(label_key)

```

Línea de Código	Explicación Técnica
def escoger_candidato(token_raw, oracion_raw, candidatos_fnames, toks_n, elegidos_file_set, elegidos_label_set, tokens_raw_list, token_index):	Define función principal de selección con múltiples parámetros de contexto
"""Selecciona UN pictograma para 'token_raw' dentro de 'oracion_raw'..."""	Documentación extensa del proceso de selección
token_n = norm(token_raw)	Normaliza el token actual

if (not INCLUIR_FUNCIONALES) and (token_n in FUNCIONALES):	Verifica si es palabra funcional y debe descartarse
return None, {"razon": "token_funcional_descartado"}	Retorna fallo por token funcional
if not candidatos_fnames:	Verifica si no hay candidatos disponibles
return None, {"razon": "sin_candidatos"}	Retorna fallo por falta de candidatos
sent_vec = get_text_vec(oracion_raw)	Obtiene embedding de la oración completa
prompt_vec = get_text_vec(f'Pictograma de {token_raw} en el contexto: {oracion_raw}')	Obtiene embedding del prompt contextualizado
tok_vec = get_text_vec(token_raw)	Obtiene embedding del token aislado
anchor = extraer_ancla_semantica(tokens_raw_list, token_index)	Busca palabra ancla semántica cercana
anchor_vec = get_text_vec(f'{token_raw} {anchor}') if anchor else None	Obtiene embedding de frase token+ancla si existe
cand_list = []	Inicializa lista de candidatos válidos
for fname in candidatos_fnames:	Itera sobre todos los candidatos posibles
if fname in elegidos_file_set:	Verifica si archivo ya fue seleccionado
continue	Salta candidato duplicado (archivo)
try: c_vec = get_image_vec(fname)	Obtiene embedding de imagen con manejo de errores
except FileNotFoundError: continue	Salta candidato si imagen no existe
label_raw = get_label(fname)	Obtiene etiqueta legible del archivo
label_n = label_norm(label_raw)	Normaliza la etiqueta

label_key = canonical_label_key_from_fname(fname)	Genera clave canónica para el concepto
if STRICT_NO_DUP and (label_key in elegidos_label_set):	Verifica duplicación de concepto si está activo
continue	Salta candidato duplicado (concepto)
cand_list.append((fname, c_vec, label_raw, label_n, label_key))	Agrega candidato válido a la lista
if not cand_list and FALLBACK_IF_ALL_DUP:	Verifica si no hay candidatos y fallback está activo
for fname in candidatos_fnames:	Reitera sobre candidatos para fallback
if fname in elegidos_file_set: continue	Mantiene restricción de archivo único
try: c_vec = get_image_vec(fname)	Obtiene embedding de imagen
except FileNotFoundError: continue	Maneja error de archivo
label_raw = get_label(fname)	Obtiene etiqueta
label_n = label_norm(label_raw)	Normaliza etiqueta
label_key = canonical_label_key_from_fname(fname)	Obtiene clave canónica
cand_list.append((fname, c_vec, label_raw, label_n, label_key))	Agrega candidato a lista de fallback
if not cand_list: return None, {"razon": "sin_candidatos_validos"}	Retorna fallo si no hay candidatos tras fallback
scored = []	Inicializa lista de candidatos puntuados
for fname, c_vec, label_raw, label_n, label_key in cand_list:	Itera sobre candidatos válidos
s_sent = float(np.dot(sent_vec, c_vec))	Calcula similitud con oración completa
s_prompt = float(np.dot(prompt_vec, c_vec))	Calcula similitud con prompt contextual
s_tok2 = float(np.dot(tok_vec, c_vec))	Calcula similitud con token aislado

s_phrase = float(np.dot(anchor_vec, c_vec)) if anchor_vec is not None else 0.0	Calcula similitud con frase ancla si existe
score = (W_SENT * s_sent) + (W_PROMPT * s_prompt) + (W_TOK2 * s_tok2)	Combina puntuaciones ponderadas
if anchor_vec is not None: score += W_PHRASE * s_phrase	Agrega componente de ancla si existe
score += reglas_heuristicas(token_raw, token_n, fname, label_raw, label_n, toks_n)	Aplica reglas heurísticas
scored.append((score, (fname, label_raw, label_key)))	Agrega candidato puntuado
ganador = pick_determinista_por_oracion(oracion_raw, scored, eps=EPS_SELECT)	Selecciona ganador con método determinista
if ganador is None: return None, {"razon": "sin_candidatos_validos"}	Maneja caso sin ganador
fname, label_raw, label_key = ganador	Desempaqueta ganador
elegidos_file_set.add(fname)	Registra archivo como seleccionado
elegidos_label_set.add(label_key)	Registra concepto como seleccionado
return {"archivo": fname, "label": label_raw}, None	Retorna resultado exitoso

Entradas:

- token_raw: Token actual a procesar (texto crudo)
- oracion_raw: Oración completa donde aparece el token
- candidatos_fnames: Lista de nombres de archivo de pictogramas candidatos
- toks_n: Lista de tokens normalizados de la oración
- elegidos_file_set: Conjunto de archivos ya seleccionados en la oración actual
- elegidos_label_set: Conjunto de conceptos ya seleccionados en la oración actual
- tokens_raw_list: Lista de tokens en crudo de la oración
- token_index: Índice del token actual en la oración

Salidas:

- Tupla con:
 - Diccionario con archivo y label del pictograma seleccionado, o None si no hay selección
 - Diccionario con razón del fallo, o None si fue exitoso

Bloque 8: Bucle Principal y Guardado

Propósito del Bloque: Ejecutar el pipeline completo de procesamiento sobre todas las oraciones, aplicar el algoritmo de selección token por token, recolectar los resultados en una estructura organizada y persistirlos en un archivo JSON junto con información de diagnóstico y vista previa.

```
# =====
# 7) Bucle principal y guardado
# =====
salidas_estructuradas = []

for item in oraciones:
    oracion = item["oracion"]
    toks    = item.get("tokens", [])
    candmap = item.get("candidatos", {})

    toks_n = tokens_norm(toks)
    elecciones = []

    # Dedupe LOCAL (cada oración es independiente)
    elegidos_file_set = set()
    elegidos_label_set = set()

    for i, tok in enumerate(toks):
        cands = candmap.get(tok, [])
        elegido, meta = escoger_candidato(
            tok, oracion, cands, toks_n,
            elegidos_file_set, elegidos_label_set,
            tokens_raw_list=toks, token_index=i
        )
        if elegido is None:
            elecciones.append(None)
        else:
            elecciones.append(elegido["archivo"])

    salidas_estructuradas.append({
        "oracion": oracion,
        "tokens": toks,
        "elecciones": elecciones,
        "secuencia_final": list(elecciones)
    })

# Guardar EXACTAMENTE como lista de objetos
with open(OUT_JSON, "w", encoding="utf-8") as f:
    json.dump(salidas_estructuradas, f, ensure_ascii=False, indent=2)

print(f"Listo: {OUT_JSON}")
print(f"Total oraciones procesadas: {len(salidas_estructuradas)}")

# Vista rápida
for x in salidas_estructuradas[:2]:
    print(json.dumps(x, ensure_ascii=False, indent=2))
```

Línea de Código	Explicación Técnica
salidas_estructuradas = []	Inicializa una lista vacía para almacenar los resultados de cada oración.
for item in oraciones:	Itera sobre cada elemento en la lista oraciones.

oracion = item["oracion"]	Extrae la cadena de texto de la oración.
toks = item.get("tokens", [])	Extrae la lista de tokens de la oración. Si no existe, usa lista vacía.
candmap = item.get("candidatos", {})	Extrae el diccionario de candidatos (token -> lista de archivos). Si no existe, usa diccionario vacío.
toks_n = tokens_norm(toks)	Normaliza los tokens de la oración para uso en heurísticas.
elecciones = []	Inicializa lista para almacenar las elecciones (archivos) de cada token en la oración actual.
elegidos_file_set = set()	Inicializa un conjunto para llevar registro de los archivos ya elegidos en la oración actual (evita duplicados de archivo).
elegidos_label_set = set()	Inicializa un conjunto para llevar registro de las etiquetas canónicas ya elegidas (evita duplicados de concepto).
for i, tok in enumerate(toks):	Itera sobre cada token en la oración, con su índice.
cands = candmap.get(tok, [])	Obtiene la lista de archivos candidatos para el token actual. Si no hay, lista vacía.
elegido, meta = escoger_candidato(...)	Llama a la función de selección con todos los parámetros necesarios.
if elegido is None:	Si no se eligió ningún pictograma para el token.
elecciones.append(None)	Agrega None a la lista de elecciones para indicar que no hay pictograma para ese token.
else:	Si se eligió un pictograma.
elecciones.append(elegido["archivo"])	Agrega el nombre del archivo del pictograma elegido.
salidas_estructuradas.append({ ... })	Agrega un diccionario con los resultados de la oración actual a la lista de salidas.
"oracion": oracion,	Incluye la oración original.
"tokens": toks,	Incluye la lista de tokens original.

"elecciones": elecciones,	Incluye la lista de elecciones (archivos o None) por token.
"secuencia_final": list(elecciones)	Incluye la misma lista de elecciones (para compatibilidad).
with open(OUT_JSON, "w", encoding="utf-8") as f:	Abre el archivo de salida para escritura con codificación UTF-8.
json.dump(salidas_estructuradas, f, ensure_ascii=False, indent=2)	Escribe la lista de resultados en el archivo JSON, con indentación y permitiendo caracteres no ASCII.
print(f"Listo: {OUT_JSON}")	Imprime la ruta del archivo de salida.
print(f"Total oraciones procesadas: {len(salidas_estructuradas)}")	Imprime el número de oraciones procesadas.
for x in salidas_estructuradas[:2]:	Itera sobre las primeras dos oraciones procesadas.
print(json.dumps(x, ensure_ascii=False, indent=2))	Imprime una representación JSON formateada de cada una.

Entradas:

- oraciones: Lista de oraciones con tokens y candidatos (del JSON cargado)
- Funciones y configuraciones de todos los bloques anteriores
- OUT_JSON: Ruta de archivo de salida definida en bloque 2

Salidas:

- salidas_estructuradas: Lista estructurada con resultados del procesamiento
- Archivo JSON en OUT_JSON con todos los resultados
- Mensajes de consola con estado del proceso
- Vista previa de las primeras 2 oraciones procesadas

Glosario y referencias

Términos clave

- Embedding: Representación vectorial de texto/imagen
- CLIP: Modelo de OpenAI para similitud texto-imagen
- Token: Unidad léxica procesable
- Deduplicación: Evitar repetición de conceptos/archivos
- Heurística: Regla basada en experiencia para mejorar resultados

Referencias

- [Sentence Transformers Documentation] (<https://www.sbert.net/>)
- [Modelos CLIP en Hugging Face] (<https://huggingface.co/models?search=clip>)
- [Documentación Pillow] (<https://pillow.readthedocs.io/>)

Este manual técnico documenta el sistema de selección de pictogramas multimodal versión 1.0.